

Лабораторная работа №1

Блочный шифр ГОСТ 28147-89: режимы работы и имитовставка

Пояснительная записка к защите. Реализация: lab1/src, язык Rust

Содержание

1. Что сделано в работе	2
2. Аббревиатуры и обозначения	2
3. Паспорт алгоритма	3
4. Устройство шифра	3
4.1. Сеть Фейстеля	3
4.2. Раундовая функция	4
4.3. Узлы замены	4
4.4. Ключевое расписание	5
5. Режимы работы	5
5.1. Простая замена (ECB)	5
5.2. Гаммирование (CTR)	6
5.3. Гаммирование с обратной связью (CFB)	6
5.4. Выработка имитовставки	7
5.5. Сводная таблица режимов	7
6. Реализация	8
6.1. Структура проекта	8
6.2. Формат шифртекста	8
6.3. Порядок байтов	8
6.4. Работа с файлами	8
7. Проверка работоспособности	8
8. Вопросы на защите	8

1. Что сделано в работе

Реализован симметричный блочный шифр ГОСТ 28147-89 и три режима его применения, а также выработка имитовставки:

Пункт стандарта	Режим	Методы <code>Gost28147_89</code>
п. 2.1	Простая замена (ECB)	<code>encrypt_ecb</code> / <code>decrypt_ecb</code>
п. 2.2	Гаммирование (CTR)	<code>encrypt_ctr</code> / <code>decrypt_ctr</code>
п. 2.3	Гаммирование с обратной связью (CFB)	<code>encrypt_cfm</code> / <code>decrypt_cfm</code>
п. 2.4	Выработка имитовставки	<code>compute_mac</code>

Программа работает и с текстом, вводимым с клавиатуры, и с файлами произвольного содержания; ключ можно менять во время работы. Криптографическое ядро лежит в `lab1/src/encryption/gost28147_89.rs`, текстовое меню – общий для обеих работ модуль `common/src/cli.rs`, точка входа с описанием режимов – `lab1/src/main.rs`.

2. Аббревиатуры и обозначения

Первое, что обычно спрашивают, – как расшифровываются названия режимов.

Сокращение	Расшифровка	Значение
ГОСТ	Государственный стандарт	Межгосударственный стандарт. Полное имя: ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования».
ECB	Electronic Codebook	«Электронная кодовая книга», в стандарте – режим простой замены . Каждый блок шифруется независимо, как будто по словарю «блок в блок».
CTR	Counter	Режим счётчика. В стандарте – режим гаммирования : шифр вырабатывает псевдослучайную последовательность (гамму), которая накладывается на текст.
CFB	Cipher Feedback	Обратная связь по шифртексту, в стандарте – гаммирование с обратной связью . В коде обозначен как <code>CFM</code> (cipher feedback mode).
MAC	Message Authentication Code	Код аутентификации сообщения; в стандарте – имитовставка . Короткий тег, вычисляемый на ключе и защищающий целостность.
IV / SV	Initialization Vector / Starting Value	Вектор инициализации, в стандарте – синхропосылка <i>S</i> . Несекретное, но неповторяющееся значение, с которого стартует режим.
S-блок	Substitution box	Узел замены: таблица подстановки. В ГОСТе восемь узлов, каждый переводит 4 бита в 4 бита.
LE	Little-endian	Порядок байтов «младший первым». В реализации все 32-битные слова собираются из байтов именно так.

Математические обозначения, используемые дальше:

Запись	Смысл
--------	-------

$X = X_1 \parallel \dots \parallel X_n$	открытый текст, разбитый на блоки по 64 бита; последний блок может быть неполным
Y_i	блок шифртекста
$K = (K_0, \dots, K_7)$	ключ: восемь 32-битных подключей, всего 256 бит
$a \oplus b$	побитовое исключающее ИЛИ (XOR)
$a \boxplus b$	сложение по модулю 2^{32}
$a \boxplus_{2^{32}-1} b$	сложение по модулю $2^{32} - 1$ (с циклическим переносом)
$\text{ROL}_{11}(a)$	циклический сдвиг 32-битного слова влево на 11 разрядов
$E_K(\cdot), E_K^{-1}(\cdot)$	зашифрование и расшифрование одного блока на ключе K
$ X $	длина сообщения в байтах

3. Паспорт алгоритма

Параметр	Значение
Год, статус	1989; с 2016 заменён на ГОСТ Р 34.12-2015 («Магма» – тот же шифр с фиксированными S-блоками и порядком байтов, плюс новый шифр «Кузнечик»)
Тип	симметричный блочный шифр
Структура	сбалансированная сеть Фейстеля
Размер блока	64 бита (8 байт)
Размер ключа	256 бит (32 байта)
Число раундов	32 (для имитовставки – 16)
Ключевое расписание	тривиальное: ключ режется на восемь 32-битных подключей, которые повторяются
Нелинейность	восемь узлов замены 4 бита в 4 бита
Полный перебор ключа	2^{256} вариантов – практически недостижимо

Почему ключ 256 бит, а блок всего 64

Длина ключа отвечает за стойкость к перебору, длина блока – за то, как быстро начнут повторяться блоки шифртекста. При 64-битном блоке по парадоксу дней рождения совпадения становятся вероятны уже после 2^{32} блоков (примерно 32 ГБ) на одном ключе. Это, а не длина ключа, – главное ограничение ГОСТ 28147-89 и причина, по которой в 2015 году появился «Кузнечик» со 128-битным блоком.

4. Устройство шифра

4.1. Сеть Фейстеля

64-битный блок делится на две 32-битные половины N_1 (младшая, первые четыре байта) и N_2 . Один раунд:

$$\left(N_1^{(i+1)}, N_2^{(i+1)} \right) = \left(N_2^{(i)} \oplus f\left(N_1^{(i)}, K_i\right), N_1^{(i)} \right)$$

То есть: к младшей половине применяется раундовая функция, результат складывается по XOR со старшей половиной, после чего половины меняются местами. После 32-го раунда перестановка не выполняется – в коде это сделано тем, что цикл всегда меняет половины, а в конце они возвращаются на место:

```
fn round(a: u32, b: u32, subkey: u32) -> (u32, u32) {
    return (b ^ Self::f(a, subkey), a);
}

fn encrypt_block(mut a: u32, mut b: u32, key: &Key<8>, encrypt: bool) -> (u32, u32) {
    for iteration in 0..ITERATIONS {
        (a, b) = Self::round(a, b, Self::get_subkey(&key, iteration, encrypt));
    }

    return (b, a);
}
```

Главное свойство сети Фейстеля

Раунд обратим **при любой** функции f : зная $(N_1^{(i+1)}, N_2^{(i+1)})$, восстанавливаем $N_1^{(i)} = N_2^{(i+1)}$ и $N_2^{(i)} = N_1^{(i+1)} \oplus f(N_2^{(i+1)}, K_i)$. Поэтому f не обязана быть биекцией, а расшифрование – это та же самая программа, в которую подклочи подаются в обратном порядке. В коде отдельной функции расшифрования вообще нет: `encrypt_block` принимает флаг `encrypt`, который влияет только на порядок подклочей.

4.2. Раундовая функция

$$f(a, K) = \text{ROL}_{11}(S(a \boxplus K))$$

Три шага:

1. **Сложение с подклочом** по модулю 2^{32} – вносит зависимость от ключа и является нелинейной операцией относительно XOR (в этом смысл выбора именно сложения, а не XOR).
2. **Замена S** : 32-битное слово режется на восемь тетрад (полубайтов), j -я тетрада заменяется по своему узлу замены S_j . Это единственный источник нелинейности относительно арифметики.
3. **Циклический сдвиг** влево на 11 бит – перемешивание: результат замены «размазывается» так, чтобы биты одной тетрады на следующем раунде попали в разные узлы замены (диффузия).

```
fn f(half: u32, subkey: u32) -> u32 {
    let sum = half.wrapping_add(subkey);
    let mut result: u32 = 0;
    for i in 0..8 {
        let four_bits = (sum & FOUR_BIT_MASKS[i]) >> (i * 4);
        let new_four_bits: u32 = S_BLOCKS[i][four_bits as usize] << (i * 4);

        result |= new_four_bits;
    }

    return result.rotate_left(11);
}
```

4.3. Узлы замены

Восемь таблиц по 16 значений, узел S_0 обслуживает младшую тетраду. Каждая таблица – перестановка: значения 0..15 встречаются ровно по одному разу, иначе замена была бы необратимой и теряла бы информацию.

Почему S-блоки не заданы в самом стандарте

ГОСТ 28147-89 оставляет узлы замены на усмотрение ведомства: они считались долговременным секретным ключевым элементом, общим для сети связи. Это даёт дополнительные примерно 354 бита «ключа», но ломает совместимость и мешает публичному анализу. Позже для открытого применения были опубликованы конкретные наборы (например, из ГОСТ Р 34.11-94,

id-GostR3411-94-TestParamSet – он и зашит в константу S_BLOCKS), а в ГОСТ Р 34.12-2015 таблица зафиксирована окончательно.

4.4. Ключевое расписание

Расписание тривиально: 256-битный ключ режется на восемь слов $K_0, \dots, K_7$ (слово j – байты $4j \dots 4j + 4$, little-endian), никаких преобразований над ними не делается. Различаются только порядки использования:

Цикл	Раунды	Порядок подключей
32-3 (зашифрование)	1–32	$K_0 \dots K_7, K_0 \dots K_7, K_0 \dots K_7, K_7 \dots K_0$
32-P (расшифрование)	1–32	$K_0 \dots K_7, K_7 \dots K_0, K_7 \dots K_0, K_7 \dots K_0$
16-3 (IMITOVSTAVKA)	1–16	$K_0 \dots K_7, K_0 \dots K_7$

Порядок 32-P – это в точности 32-3, прочитанный задом наперёд; именно поэтому одна и та же сеть расшифровывает.

```
fn get_subkey(key: &Key<8>, idx: usize, encrypt: bool) -> u32 {
    if encrypt {
        if idx < 24 { return key[idx % 8]; } else { return key[(32 - (idx + 1)) % 8]; }
    } else {
        if idx < 8 { return key[idx]; } else { return key[7 - idx % 8]; }
    }
}
```

Слабое место

Простота расписания – известная претензия к ГОСТу: подключи не зависят от номера раунда и повторяются восьмёрками, что делает шифр уязвимым к атакам на связанных ключах и к «отражающим» (reflection) атакам. Лучшие опубликованные результаты (Исобеэ, 2011; Динур–Данкельман–Шамир, 2011) снижают сложность восстановления ключа примерно до 2^{192} – теоретический результат, на практике неприменимый, но формально шифр уже не «256-битный».

5. Режимы работы

Шифр умеет преобразовывать ровно один 64-битный блок. Режим – это правило, как применить это преобразование к сообщению произвольной длины.

5.1. Простая замена (ЕСВ)

$$Y_i = E_K(X_i), \quad X_i = E_K^{-1}(Y_i), \quad i = 1, \dots, n$$

Каждый блок обрабатывается независимо от соседей. Синхропосылки нет.

Дополнение. Стандарт требует сообщение, кратное 8 байтам. В реализации используется дополнение нулями, а число добавленных байтов дописывается одним байтом в конец шифртекста:

$$|Y| = 8 \cdot \lceil |X| / 8 \rceil + 1$$

Служебный байт нужен потому, что иначе расшифрованный текст нельзя отличить от текста, который действительно кончался нулями. При расшифровании этот байт снимается первым, и проверяется, что он меньше 8, а остаток кратен размеру блока (`check_padded_blocks` в `lab1/src/main.rs`).

Свойства. Одинаковым блокам открытого текста соответствуют одинаковые блоки шифртекста – шифртекст протекает структуру данных (классическая иллюстрация – «зашифрованный пингвин Тух», на котором сквозь ЕСВ видно исходное изображение). Блоки можно переставить, удалить или продублировать, и расшифрование этого не заметит. Зато режим полностью распараллеливается и допускает произвольный доступ к любому блоку.

Ошибка в одном бите шифртекста портит весь свой блок (8 байт) и только его.

5.2. Гаммирование (CTR)

Шифр используется не для преобразования текста, а для выработки **гаммы** – псевдослучайной последовательности, которая накладывается на текст по XOR.

$$\begin{aligned}(N_1, N_2) &= E_K(S) \\ N_1^{(i)} &= N_1^{(i-1)} \boxplus C_2, \quad C_2 = 01010101_{16} \\ N_2^{(i)} &= N_2^{(i-1)} \boxplus_{2^{32}-1} C_1, \quad C_1 = 01010104_{16} \\ \Gamma_i &= E_K(N_1^{(i)}, N_2^{(i)}) \\ Y_i &= X_i \oplus \Gamma_i\end{aligned}$$

Расшифрование – та же формула: $X_i = Y_i \oplus \Gamma_i$. Блочное расшифрование E_K^{-1} в этом режиме не используется вообще.

Константы. C_1 и C_2 подобраны так, чтобы счётчик при любом старте проходил длинный цикл без повторов: N_1 наращивается по модулю 2^{32} , а N_2 – по модулю $2^{32} - 1$ (сложение с циклическим переносом), поэтому периоды двух половин взаимно просты и общий период равен $2^{32}(2^{32} - 1) \approx 2^{64}$ блоков. В коде это

```
n_1 = n_1.wrapping_add(C2);
n_2 = {
    let (sum, carry) = n_2.overflowing_add(C1);
    sum + carry as u32
};
```

Свойства. Дополнение не нужно: последний блок гаммы просто обрезается по фактической длине, поэтому $|Y| = 8 + |X|$ (восемь байт – открыто передаваемая синхропосылка). Одинаковые блоки открытого текста дают разные блоки шифртекста. Гамму можно вычислить заранее, до появления текста, и считать её параллельно; поддерживается произвольный доступ.

Искажение бита шифртекста портит **ровно тот же бит** открытого текста – ошибка не размножается. Это удобно для каналов с помехами, но означает, что противник может осмысленно менять биты сообщения, если оно не защищено имитовставкой.

Самая опасная ошибка режима гаммирования

Синхропосылка не секретна, но обязана быть **уникальной** для каждого сообщения на данном ключе. Если два сообщения зашифрованы одной парой (K, S) , то

$$Y \oplus Y' = (X \oplus \Gamma) \oplus (X' \oplus \Gamma) = X \oplus X'$$

– гамма сокращается, и противник получает XOR двух открытых текстов, из которого при известной статистике языка восстанавливаются оба. В реализации S берётся из генератора случайных чисел при каждом зашифровании (`rand : rng().random()`), 64 случайных бита.

5.3. Гаммирование с обратной связью (CFB)

Отличие от предыдущего режима в том, откуда берётся вход шифра: не из счётчика, а из уже полученного шифртекста.

$$\begin{aligned}\Gamma_1 &= E_K(S) \\ \Gamma_i &= E_K(Y_{i-1}), \quad i \geq 2 \\ Y_i &= X_i \oplus \Gamma_i, \quad X_i = Y_i \oplus \Gamma_i\end{aligned}$$

Регистр обратной связи обновляется только полным блоком; последний неполный блок гаммируется по фактической длине, и обновлять регистр уже незачем.

Свойства. Длина сохраняется, $|Y| = 8 + |X|$, дополнения нет. Блочное расшифрование снова не используется. Зашифрование строго последовательно (нужен предыдущий блок шифртекста), а расшифрование распараллеливается: все Y_{i-1} уже известны.

Распространение ошибки – главное отличие от CTR. Искажение бита в Y_i даёт: тот же искажённый бит в X_i плюс **полностью испорченный** блок X_{i+1} (потому что Y_i ушёл в шифр). Начиная с X_{i+2} расшифрование восстанавливается само – это свойство называется **самосинхронизацией**: режим переживает и потерю, и вставку целого блока, выпадает лишь один блок.

5.4. Выработка имитовставки

Имитовставка защищает **целостность и подлинность**, а не тайну: она отвечает на вопрос «это сообщение точно не изменено и выработано тем, кто знает ключ».

$$S_0 = 0$$

$$S_i = 16\text{-}3(S_{i-1} \oplus X_i), \quad i = 1, \dots, n$$

$$I_r = \text{отрезок длиной } r \text{ бит из } S_n, \quad r \leq 32$$

Сообщение предварительно дополняется нулями до кратности 8 байтам. Вместо 32 раундов используется цикл 16-3 (первые 16 раундов зашифрования), длина имитовставки задаётся пользователем, по умолчанию 32 бита.

Это CBC-MAC

Схема «XOR с предыдущим состоянием, затем зашифрование» – в точности CBC-MAC: имитовставка равна последнему блоку шифртекста в режиме сцепления блоков с нулевой синхропосылкой. Отсюда и известные свойства: усечение до r бит даёт вероятность угадать имитовставку 2^{-r} , то есть при $r = 32$ – примерно один шанс из четырёх миллиардов на попытку.

Почему 16 раундов, а не 32: имитовставку не нужно обрабатывать, от неё требуется только стойкость к подбору, а половинный цикл вдвое дешевле. Отдельная причина – разделение функций: результат 16-3 не является шифртекстом, поэтому одну и ту же схему нельзя случайно применить и для шифрования, и для аутентификации.

Какой именно отрезок

Стандарт фиксирует длину $r \leq 32$, но не то, какие именно биты блока S_n берутся. В реализации берутся младшие r бит второй половины блока (`s_2 & mac_mask`). Важно лишь, чтобы отправитель и получатель брали один и тот же отрезок. На пустом сообщении имитовставка не определена – функция возвращает `None`, и меню печатает об этом ошибку.

5.5. Сводная таблица режимов

Свойство	ECB	CTR	CFB
Нужна синхропосылка	нет	да	да
Дополнение сообщения	да, нули и байт-счётчик	нет	нет
Длина шифртекста	$8 \lceil X / 8 \rceil + 1$	$8 + X $	$8 + X $
Используется E_K^{-1}	да	нет	нет
Одинаковые блоки X	дают одинаковые Y	дают разные Y	дают разные Y
Зашифрование параллелится	да	да	нет
Расшифрование параллелится	да	да	да
Гамма заранее	–	да	нет
Ошибка в бите Y_i портит	весь блок X_i	один бит X_i	бит в X_i и весь X_{i+1}
Самосинхронизация	–	нет	да, через один блок

6. Реализация

6.1. Структура проекта

Файл	Содержимое
lab1/src/encryption/gost28147_89.rs	весь шифр: S-блоки, <code>f</code> , <code>round</code> , <code>encrypt_block</code> , <code>mac_block</code> , четыре режима
lab1/src/main.rs	описание режимов для меню и проверки длины шифр-текста
common/src/cli.rs	текстовое меню, ввод/вывод, hex, разбор ключа – общее с работой №2
common/src/key.rs	тип <code>Key<8></code> – восемь 32-битных слов

Меню знает о режимах только то, что каждый из них – пара функций `fn(Vec<u8>, &Key<8>) -> Vec<u8>` плюс необязательная проверка входа. Поэтому один и тот же CLI обслуживает и ГОСТ, и BelT из работы №2.

6.2. Формат шифртекста

Стандарт считает синхропосылку внешним параметром протокола. В работе принято соглашение: она вырабатывается случайно при каждом зашифровании и передаётся открыто первым блоком.

Режим	Шифртекст
ECB	блоки по 8 байт, затем 1 байт с числом добавленных нулей
CTR, CFB	S (8 байт), затем гамма-поток ($ X $ байт)

6.3. Порядок байтов

Все 32-битные слова – и половины блока, и подключи – собираются из байтов как little-endian (`u32::from_le_bytes`). Это соглашение выдержано и при разборе ключа из hex, поэтому введённый ключ читается байт за байтом слева направо.

6.4. Работа с файлами

В файловом режиме программа запрашивает путь к исходному файлу и к файлу результата. По умолчанию при зашифровании добавляется расширение `.gost`, при расшифровании оно снимается; существующий файл перезаписывается только после подтверждения. Перед расшифрованием вызывается проверка, соответствует ли длина шифртекста выбранному режиму, – так пользователь получает внятную ошибку вместо мусора на выходе.

7. Проверка работоспособности

1. Круговой прогон: для всех трёх режимов и разных длин сообщения расшифрование восстанавливает исходные байты; файлы сравнивались побайтово.
2. ECB: текст из повторяющихся символов даёт шифртекст с повторяющимися блоками – наглядно видно, что режим протекает структуру.
3. CTR и CFB: два зашифрования одного и того же текста дают разные шифртексты, потому что синхропосылка каждый раз новая.
4. Распространение ошибки: инверсия одного бита шифртекста портит один бит (CTR) либо бит и весь следующий блок (CFB).
5. Имитовставка: изменение любого бита сообщения меняет имитовставку.

8. Вопросы на защите

? Почему сеть Фейстеля позволяет расшифровывать той же схемой?

Раунд устроен так, что из выхода однозначно восстанавливается вход при любой функции f : одна

половина проходит через раунд без изменений и служит «подсказкой» для обращения второй. Поэтому расшифрование – это те же 32 раунда с подключами в обратном порядке, а сама f не обязана быть обратимой.

? Что будет, если при расшифровании подать подключи в прямом порядке?

Получится не исходный текст, а другое псевдослучайное значение. Порядок 32-Р – это зеркало 32-З, только оно «раскручивает» раунды.

? Зачем в раундовой функции сразу и сложение, и XOR?

Они нелинейны друг относительно друга: сложение по модулю 2^{32} переносит разряды, XOR – нет. Смешение двух операций из разных алгебраических групп – дешёвый источник нелинейности, дополняющий узлы замены.

? Почему сдвиг именно на 11 бит?

11 не кратно 4, поэтому после сдвига биты одной тетрады попадают в разные узлы замены следующего раунда. Это обеспечивает диффузию: изменение одного бита входа за несколько раундов затрагивает все биты блока.

? Почему ЕСВ нельзя применять к реальным данным?

Он детерминирован и поблочен: одинаковые блоки дают одинаковый шифртекст, так что видны повторы, структура записей, границы полей. Кроме того, шифртекст можно переставлять и склеивать из кусков разных сообщений – расшифрование этого не заметит. Допустимое применение – шифрование одиночных случайных блоков, например ключей.

? Зачем в ЕСВ отдельный байт с числом дополненных нулей?

Дополнение нулями неоднозначно: текст, который сам кончался нулями, невозможно отличить от дополненного. Байт-счётчик снимает неоднозначность. Цена – шифртекст на байт длиннее кратности блока.

? Должна ли синхропосылка быть секретной?

Нет. Она должна быть **неповторяющейся** при данном ключе (для гаммирования – желательно ещё и непредсказуемой). Поэтому её передают открыто, первым блоком шифртекста.

? Что произойдёт, если повторить синхропосылку в режиме гаммирования?

Гамма повторится, и XOR двух шифртекстов даст XOR двух открытых текстов: $Y \oplus Y' = X \oplus X'$. Ключ при этом не нужен, оба сообщения восстанавливаются статистически. Это отказ защиты целиком.

? Почему в гамма-режимах не нужно дополнение?

Текст не пропускается через блочное преобразование, а складывается по XOR с гаммой. Лишние байты гаммы просто отбрасываются, поэтому длина шифртекста равна длине открытого текста.

? Почему в CTR и CFB не используется расшифрование блока?

Шифр вырабатывает гамму, и на зашифровании, и на расшифровании – одну и ту же. Обратное преобразование E_K^{-1} не нужно; в аппаратной реализации это экономит половину схемы.

? Чем гаммирование по ГОСТу отличается от классического CTR?

Идея та же, отличается способ наращивания счётчика: вместо инкремента на единицу используются константы $C_1 = 01010104_{16}$ и $C_2 = 01010101_{16}$, причём младшая половина наращивается по модулю 2^{32} , а старшая – по модулю $2^{32} - 1$. Взаимно простые периоды дают общий период около 2^{64} блоков.

? В чём практическая разница между CTR и CFB?

В распространении ошибок и в параллелизме. CTR: ошибка не размножается, обе операции параллелятся, гамму можно готовить заранее. CFB: ошибка в блоке портит и следующий блок, зашифрование последовательно, зато режим самосинхронизируется после потери или вставки блока.

? Что значит «самосинхронизация»?

Состояние режима – это последние переданные блоки шифртекста. Если один блок потерян или искажён, через один блок регистр обратной связи снова заполнен корректными данными, и расшифрование продолжается правильно. В CTR такого нет: сбившийся счётчик разрушает всё до конца сообщения.

? Зачем нужна имитовставка, если данные уже зашифрованы?

Шифрование даёт конфиденциальность, но не целостность. В гамма-режимах противник может инвер-

тировать выбранные биты шифртекста и предсказуемо изменить открытый текст, не зная ключа. Имитовставка ловит любое такое изменение, потому что вычисляется на секретном ключе.

? Почему нельзя вместо имитовставки взять CRC или хэш без ключа?

CRC линейна: изменив сообщение, противник пересчитает контрольную сумму. Хэш без ключа пересчитывается кем угодно. Имитовставка требует знания ключа, поэтому подделать её без ключа нельзя.

? Почему для имитовставки используются 16 раундов, а не 32?

Имитовставку не нужно обрабатывать, от неё требуется только стойкость к подбору, а половинный цикл вдвое дешевле. Одновременно это разделяет функции: результат 16-3 не является шифртекстом, и одно и то же преобразование нельзя случайно использовать для двух разных целей.

? Какова вероятность подделать имитовставку длиной r бит?

При случайном угадывании – 2^{-r} на одну попытку; при $r = 32$ это примерно $2.3 \cdot 10^{-10}$. Отсюда и требование ограничивать число попыток проверки.

? Сначала шифровать, потом считать имитовставку, или наоборот?

Современная рекомендация – encrypt-then-MAC: имитовставка считается от шифртекста, тогда получатель отбрасывает подделку, вообще не расшифровывая её. ГОСТ 28147-89 описывает выработку имитовставки от открытого текста (MAC-then-encrypt), что само по себе работает, но требует аккуратности в протоколе.

? Почему узлы замены не заданы в стандарте и что это меняет?

Они считались долговременным секретным ключевым элементом ведомственной сети. Плюс – дополнительная неизвестная противнику информация; минусы – нет совместимости между реализациями, затруднён открытый анализ, а неудачно выбранная таблица может ослабить шифр. В ГОСТ Р 34.12-2015 таблицу зафиксировали.

? Известны ли атаки на ГОСТ 28147-89?

Практических – нет, полный перебор 2^{256} недостижим. Теоретически шифр ослаблен простым ключевым расписанием: в 2011 году были опубликованы атаки (Исобеэ; Динур–Данкельман–Шамир) со сложностью порядка 2^{192} операций, что формально ниже заявленных 256 бит. Отдельно критикуется 64-битный блок: после примерно 2^{32} блоков на одном ключе начинаются коллизии блоков.

? Чем ГОСТ 28147-89 заменён?

ГОСТ Р 34.12-2015 определяет два шифра: «Магма» – тот же алгоритм с фиксированными S-блоками и уточнённым порядком байтов, и «Кузнечик» – новый SP-шифр со 128-битным блоком и 256-битным ключом.

? Как в программе выбирается длина имитовставки и почему не больше 32 бит?

Длина запрашивается в меню, по умолчанию 32. Верхняя граница – половина блока: имитовставка берётся из 64-битного состояния, и стандарт ограничивает её 32 битами.